Deep Learning

Foundations: Neural Networks to Representation Learning

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-II





Outline

1. Deep Learning Foundations
2. Convolutional Neural Networks
3. Sequence Models & Transformers
4. Unsupervised & Representation Learning





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1 Deep Learning Foundations

2 Convolutional Neural Networks

3 Sequence Models & Transformers

4 Unsupervised & Representation Learning

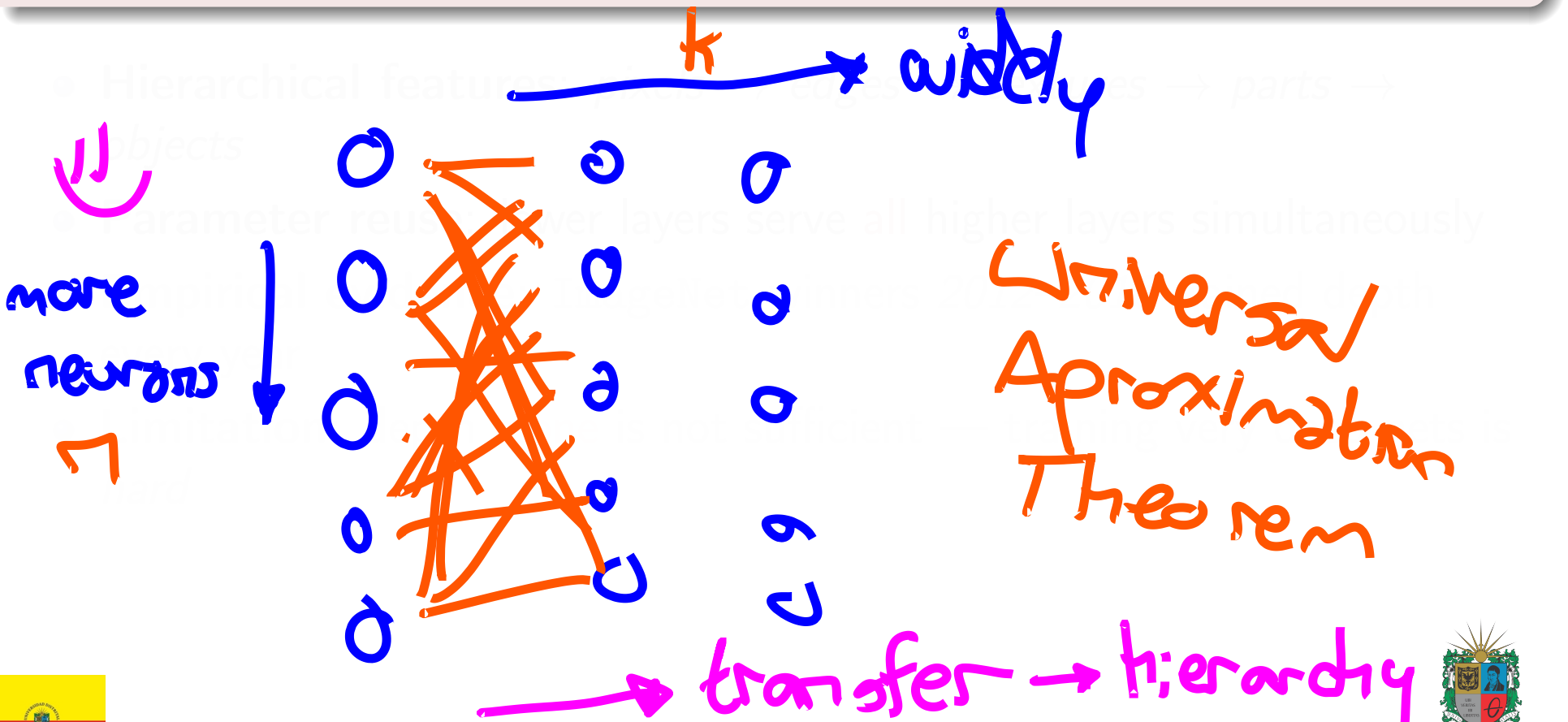




Why Depth?

Exponential Representational Efficiency

Functions representable by a depth-k network with n neurons per *layer* require exponentially more neurons in a *depth-1 network*.



Why Depth?

Exponential Representational Efficiency

Functions representable by a depth-k network with n neurons per *layer* require exponentially more neurons in a *depth-1 network*.

- **Hierarchical features**: *pixels* $\rightarrow$ *edges* $\rightarrow$ *textures* $\rightarrow$ *parts* $\rightarrow$ *objects*



- **Parameter reuse**: lower layers serve all higher layers simultaneously
- **Empirical evidence**: ImageNet winners *2012–2016* gained depth every year
- **Limitation**: depth alone is not sufficient; training very deep nets is *hard*





Why Depth?

Exponential Representational Efficiency

Functions representable by a depth-k network with n neurons per *layer* require exponentially more neurons in a *depth-1 network*.

- **Hierarchical features**: *pixels* $\rightarrow$ *edges* $\rightarrow$ *textures* $\rightarrow$ *parts* $\rightarrow$ *objects*
- **Parameter reuse**: lower layers serve all higher layers simultaneously
- **Empirical evidence**: `ImageNet` winners *2012–2016* gained depth every year
- **Limitation**: depth alone is not sufficient — training very deep nets is *hard*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Why Depth?

Exponential Representational Efficiency

Functions representable by a depth-k network with n neurons per *layer* require exponentially more neurons in a *depth-1 network*.

- **Hierarchical features**: *pixels* $\rightarrow$ *edges* $\rightarrow$ *textures* $\rightarrow$ *parts* $\rightarrow$ *objects*
- **Parameter reuse**: lower layers serve all higher layers simultaneously
- **Empirical evidence**: `ImageNet` winners *2012–2016* gained depth every year
- **Limitation**: depth alone is not sufficient — training very deep nets is *hard*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Why Depth?

Exponential Representational Efficiency

Functions representable by a depth-k network with n neurons per *layer* require exponentially more neurons in a *depth-1 network*.

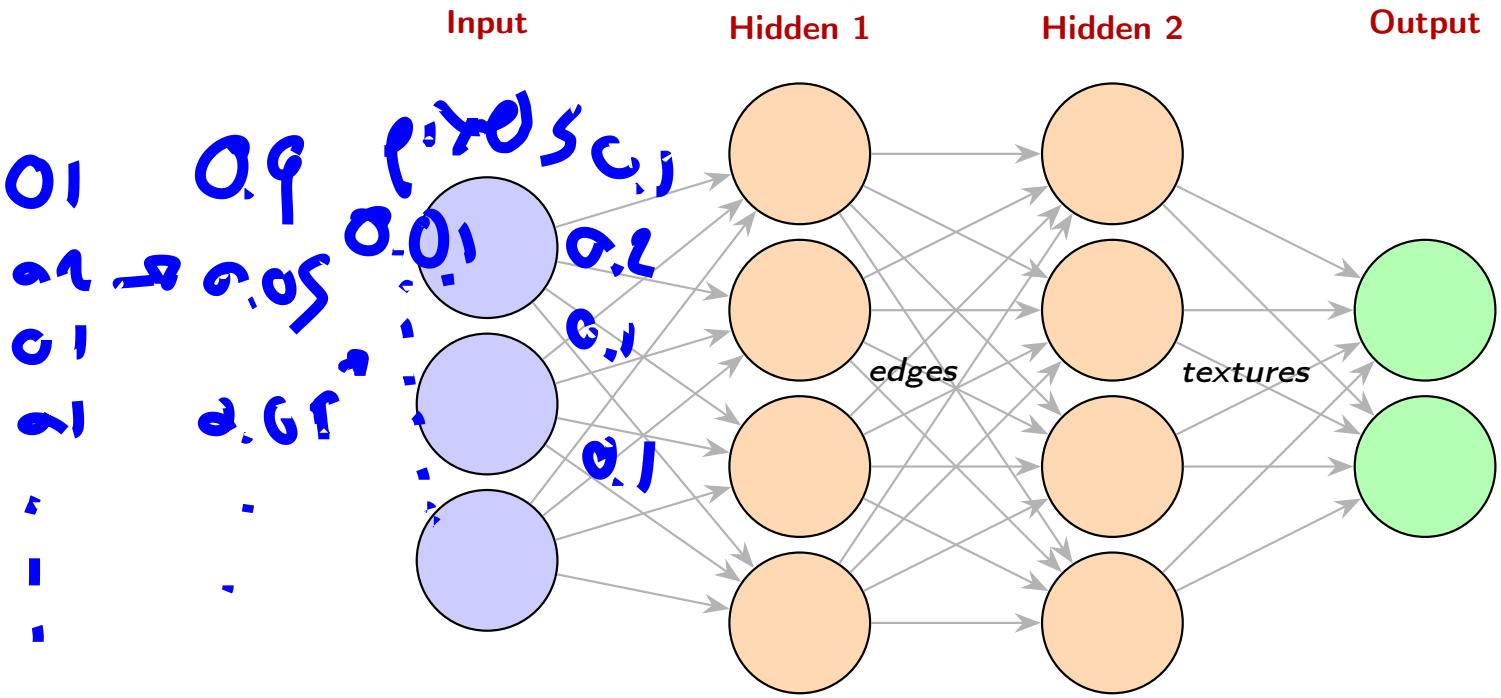
- **Hierarchical features**: *pixels* $\rightarrow$ *edges* $\rightarrow$ *textures* $\rightarrow$ *parts* $\rightarrow$ *objects*
- **Parameter reuse**: lower layers serve all higher layers simultaneously
- **Empirical evidence**: `ImageNet` winners *2012–2016* gained depth every year
- **Limitation**: depth alone is not sufficient — ~~training~~ very deep nets is *hard*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Neural Network Depth: Visual Intuition

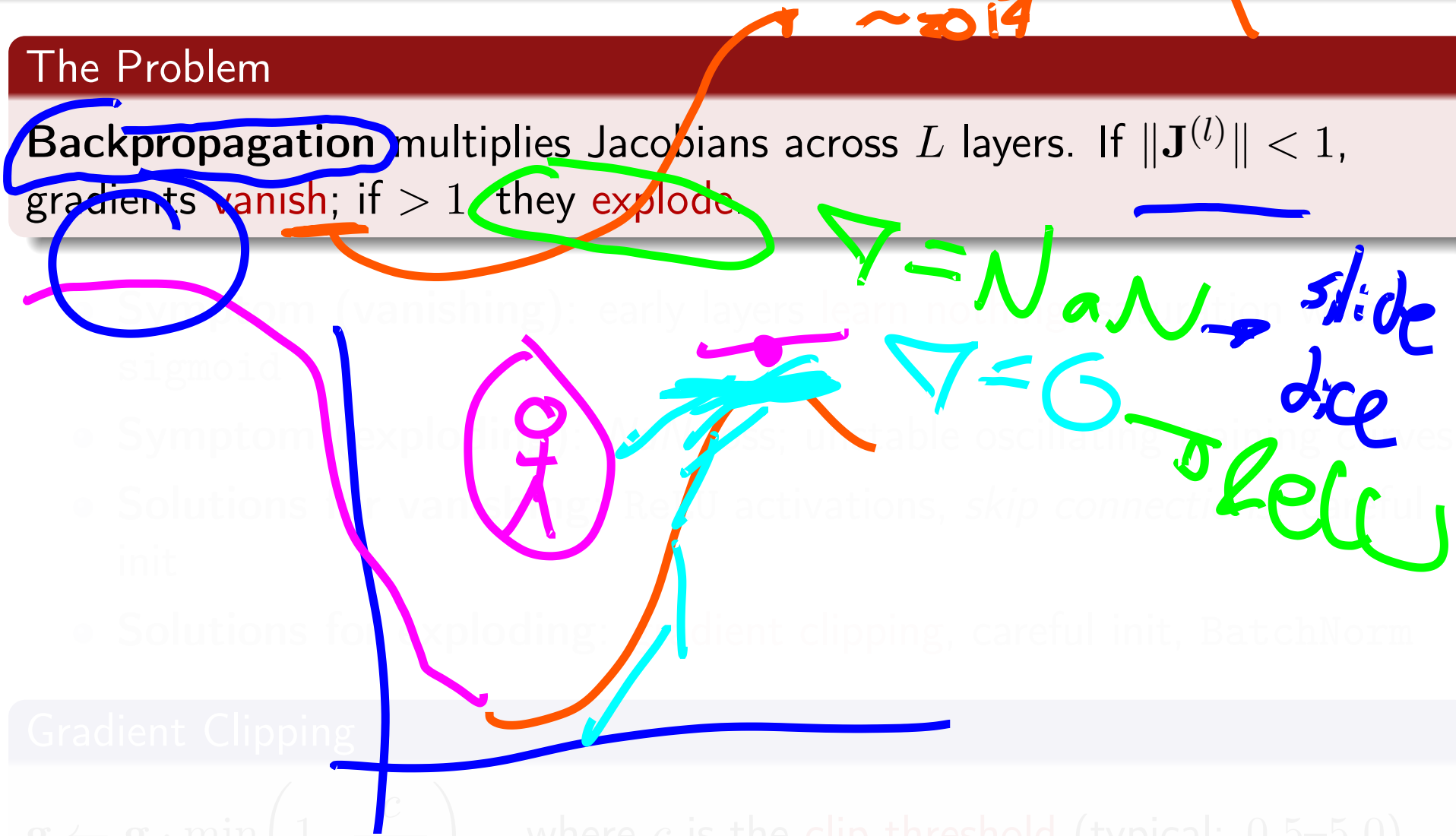


Each layer learns more abstract representations.

Vanishing and Exploding Gradients

The Problem

Backpropagation multiplies Jacobians across L layers. If $\|\mathbf{J}^{(l)}\| < 1$, gradients vanish; if > 1, they explode.



Gradient Clipping

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right)$$ where c is the clip threshold (typical: 0.5–5.0)

Vanishing and Exploding Gradients

The Problem

Backpropagation multiplies Jacobians across L layers. If $\|\mathbf{J}^{(l)}\| < 1$, gradients vanish; if > 1, they explode.

- **Symptom (vanishing)**: early layers learn nothing; saturation with `sigmoid`
- **Symptom (exploding)**: *NaN* loss; unstable oscillating training curves
- **Solutions for vanishing**: `ReLU` activations, *skip connections*, careful init
- **Solutions for exploding**: gradient clipping, careful init, `BatchNorm`

Gradient Clipping

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right)$$ where c is the clip threshold (typical: 0.5–5.0)

Vanishing and Exploding Gradients

The Problem

Backpropagation multiplies Jacobians across L layers. If $\|\mathbf{J}^{(l)}\| < 1$, gradients vanish; if > 1, they explode.

- **Symptom (vanishing)**: early layers learn nothing; saturation with `sigmoid`
- **Symptom (exploding)**: *NaN* loss; unstable oscillating training curves
- **Solutions for vanishing**: `ReLU` activations, *skip connections*, careful init
- **Solutions for exploding**: gradient clipping, careful init, `BatchNorm`

Gradient Clipping

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right)$$ where c is the clip threshold (typical: 0.5–5.0)

Vanishing and Exploding Gradients

The Problem

Backpropagation multiplies Jacobians across L layers. If $\|\mathbf{J}^{(l)}\| < 1$, gradients vanish; if > 1, they explode.

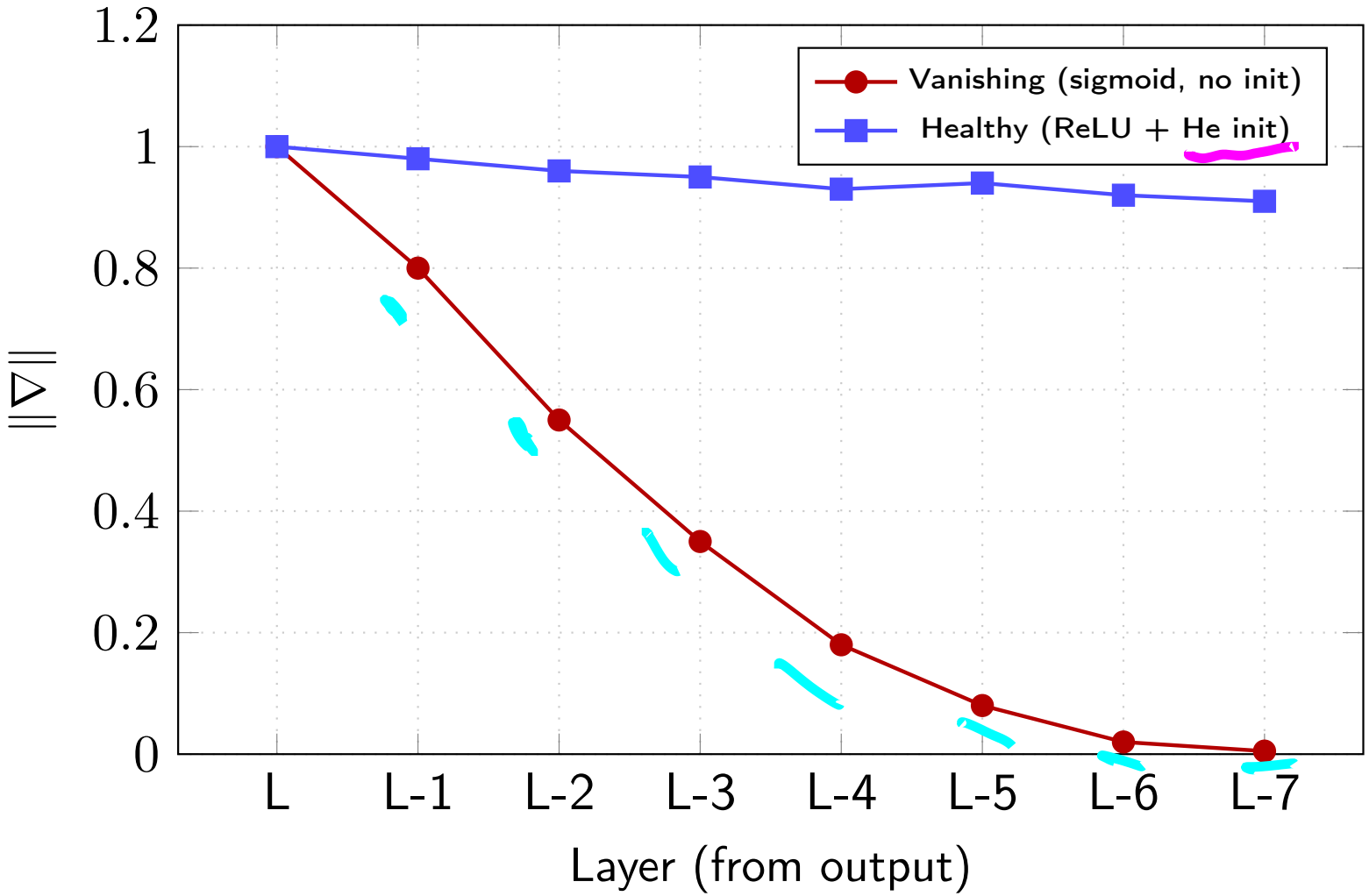
- **Symptom (vanishing)**: early layers learn nothing; saturation with `sigmoid`
- **Symptom (exploding)**: *NaN* loss; unstable oscillating training curves
- **Solutions for vanishing**: `ReLU` activations, *skip connections*, careful init
- **Solutions for exploding**: gradient clipping, careful init, `BatchNorm`

Gradient Clipping

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right)$$

where c is the clip threshold (typical: 0.5–5.0)

Gradient Magnitude Across Layers



Vanishing: gradients collapse to zero far from the output.

Weight Initialization

`Xavier / Glorot`

$$W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

- Designed for `tanh` / `sigmoid`
- Keeps variance stable through layers

`He / Kaiming`

$$W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}}\right)$$

- Designed for `ReLU`
- Accounts for *half the units* being zero
- Default for modern networks

Zero Init Kills Learning

If all weights are *zero*, all neurons compute identical gradients — *symmetry* never breaks. Always break symmetry with *random initialisation*.





Weight Initialization

`Xavier / Glorot`

$$W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

- Designed for `tanh` / `sigmoid`
- Keeps variance stable through layers

`He / Kaiming`

$$W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}}\right)$$

- Designed for `ReLU`
- Accounts for *half the units* being zero
- Default for modern networks

Zero Init Kills Learning

If all weights are *zero*, all neurons compute identical gradients — *symmetry* never breaks. Always break symmetry with *random initialisation*.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

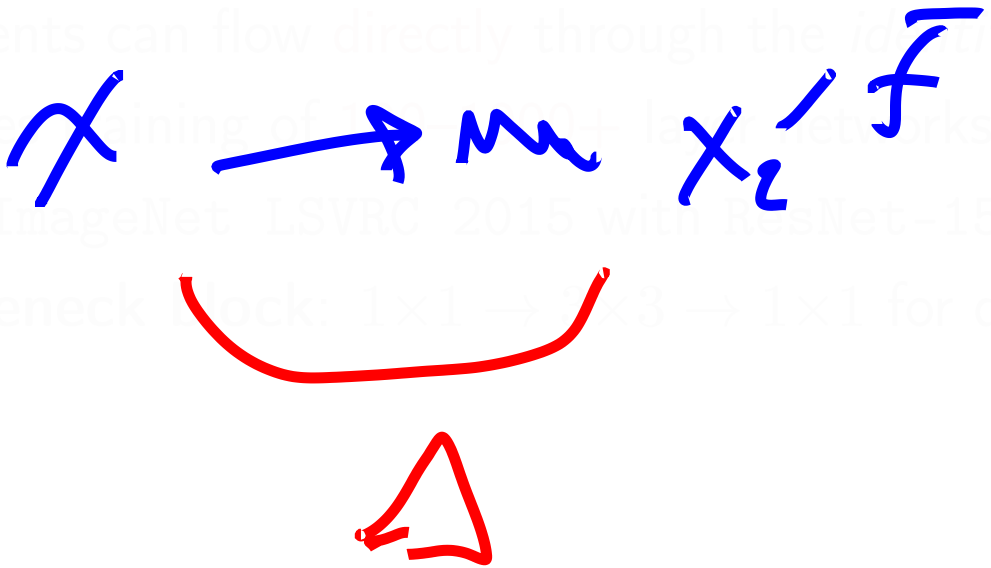
Residual Connections (ResNet)

Skip Connection (He *et al.*, 2016)

Instead of learning $\mathcal{H}(\mathbf{x})$, learn the residual:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \mathbf{W}) + \mathbf{x}$$

where $\mathcal{F}$ is a stack of conv + BN + ReLU layers.



Residual Connections (ResNet)

Skip Connection (He *et al.*, 2016)

Instead of learning $\mathcal{H}(\mathbf{x})$, learn the residual:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \mathbf{W}) + \mathbf{x}$$

where $\mathcal{F}$ is a stack of conv + BN + ReLU layers.

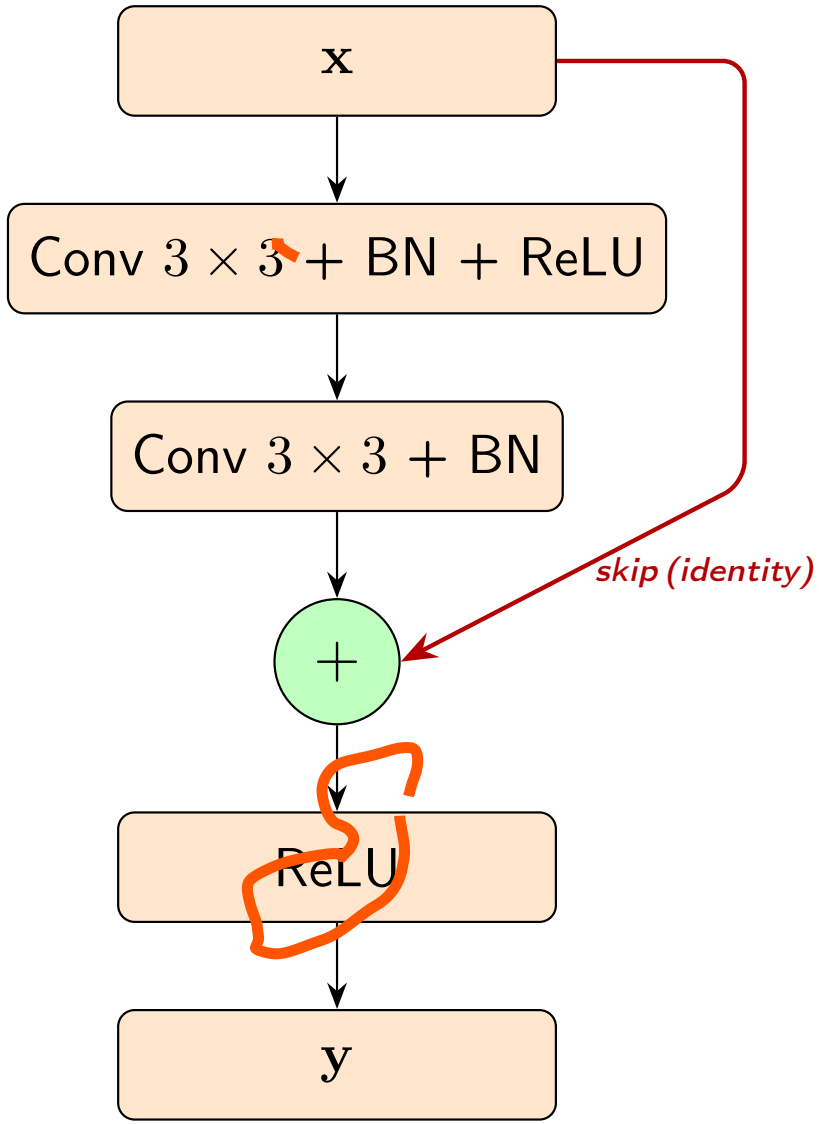
- Gradients can flow directly through the *identity shortcut*
- Enables training of 100–1000+ layer networks
- Won ImageNet LSVRC 2015 with ResNet-152 (3.57% top-5 error)
- **Bottleneck block**: $1\times1 \rightarrow 3\times3 \rightarrow 1\times1$ for deeper/efficient variants





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

ResNet Skip Connection: Block Diagram



Other Deep Architectures

DenseNet (Huang *et al.*, 2017)

Each layer receives feature maps from all preceding layers:

$$\mathbf{x}_l = \mathcal{H}_l([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}])$$

- Maximum gradient flow; strong feature reuse
- Fewer parameters than ResNet at similar accuracy

EfficientNet (Tan & Le, 2019)

- Compound scaling: width $\times$ depth $\times$ resolution simultaneously
- B0–B7 family covers *mobile* to *server* deployments

- These architectures are the backbones used in Challenge 7 (Transfer Learning)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Other Deep Architectures

DenseNet (Huang *et al.*, 2017)

Each layer receives feature maps from all preceding layers:

$$\mathbf{x}_l = \mathcal{H}_l([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}])$$

- Maximum gradient flow; strong feature reuse
- Fewer parameters than ResNet at similar accuracy

EfficientNet (Tan & Le, 2019)

- Compound scaling: width $\times$ depth $\times$ resolution simultaneously
- B0–B7 family covers *mobile* to *server* deployments
- These architectures are the backbones used in Challenge 7 (Transfer Learning)





Other Deep Architectures

`DenseNet` (Huang *et al.*, 2017)

Each layer receives feature maps from all preceding layers:

$$\mathbf{x}_l = \mathcal{H}_l([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}])$$

- Maximum gradient flow; strong feature reuse
- Fewer parameters than `ResNet` at similar accuracy

`EfficientNet` (Tan & Le, 2019)

- Compound scaling: width $\times$ depth $\times$ resolution simultaneously
- B0–B7 family covers *mobile* to *server* deployments

- These architectures are the backbones used in Challenge 7 (Transfer Learning)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Normalization Variants

Batch Normalization (BN)

Normalise over the batch dimension; ideal for CNNs with large batches

Layer Normalization (LN)

Normalise over the feature dimension; ideal for Transformers and RNNs; independent of batch size

Group Normalization (GN)

Normalise within groups of channels; better than BN with small batches (*object detection*, *segmentation*)

- Rule of thumb: BN for CNNs, LN for Transformers/RNNs, GN for detection heads





Normalization Variants

Batch Normalization (BN)

Normalise over the batch dimension; ideal for CNNs with large batches

Layer Normalization (LN)

Normalise over the feature dimension; ideal for Transformers and RNNs; independent of batch size

Group Normalization (GN)

Normalise within groups of channels; better than BN with small batches (*object detection*, *segmentation*)

- Rule of thumb: BN for CNNs, LN for Transformers/RNNs, GN for *detection heads*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Normalization Variants

Batch Normalization (BN)

Normalise over the batch dimension; ideal for CNNs with large batches

Layer Normalization (LN)

Normalise over the feature dimension; ideal for Transformers and RNNs; independent of batch size

Group Normalization (GN)

Normalise within groups of channels; better than BN with small batches (*object detection*, *segmentation*)

- Rule of thumb: BN for CNNs, LN for Transformers/RNNs, GN for detection heads





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Normalization Variants

Batch Normalization (BN)

Normalise over the batch dimension; ideal for CNNs with large batches

Layer Normalization (LN)

Normalise over the feature dimension; ideal for Transformers and RNNs; independent of batch size

Group Normalization (GN)

Normalise within groups of channels; better than BN with small batches (*object detection*, *segmentation*)

- **Rule of thumb**: BN for CNNs, LN for Transformers/RNNs, GN for *detection heads*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Example: Training Deep vs Shallow Networks

- **Task**: `CIFAR-10` image classification
- **Model A**: plain 20-layer `CNN` (no skip connections)
- **Model B**: `ResNet-20` (same depth, with skip connections)
- **Observation**: Model A shows higher training error than a shallower network (degradation problem); `ResNet-20` avoids this
- **Experiment**: monitor gradient norms per layer during training
- **Workshop connection**: `ResNet` backbones from this class are used directly in *Challenge 7 Part A*





Practical Example: Training Deep vs Shallow Networks

- **Task**: `CIFAR-10` image classification
- **Model A**: plain 20-layer `CNN` (no skip connections)
- **Model B**: `ResNet-20` (same depth, with skip connections)
- **Observation**: Model A shows higher training error than a shallower network (degradation problem); `ResNet-20` avoids this
- **Experiment**: monitor gradient norms per layer during training
- **Workshop connection**: `ResNet` backbones from this class are used directly in *Challenge 7 Part A*





Outline

1 Deep Learning Foundations

2 Convolutional Neural Networks

3 Sequence Models & Transformers

4 Unsupervised & Representation Learning





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Motivation: Images as Data

The Problem with Plain MLPs

A $224 \times 224 \times 3$ image has $150,528$ input dimensions. A single *fully-connected* layer with $1,000$ neurons requires $150M$ parameters.

- Local structure: nearby pixels are strongly correlated
- Translation equivariance: a feature detector should work anywhere in the image
- Hierarchical composition: objects are made of *parts*, parts of *edges*

Convolutional Layers

Exploit all three properties through weight sharing and local connectivity.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Motivation: Images as Data

The Problem with Plain MLPs

A $224 \times 224 \times 3$ image has $150,528$ input dimensions. A single *fully-connected* layer with $1,000$ neurons requires $150M$ parameters.

- **Local structure**: nearby pixels are strongly correlated
- **Translation equivariance**: a feature detector should work anywhere in the image
- **Hierarchical composition**: objects are made of *parts*, parts of *edges*

Convolutional Layers

Exploit all three properties through weight sharing and local connectivity.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Motivation: Images as Data

The Problem with Plain MLPs

A $224 \times 224 \times 3$ image has $150,528$ input dimensions. A single *fully-connected* layer with $1,000$ neurons requires $150M$ parameters.

- **Local structure**: nearby pixels are strongly correlated
- **Translation equivariance**: a feature detector should work anywhere in the image
- **Hierarchical composition**: objects are made of *parts*, parts of *edges*

Convolutional Layers

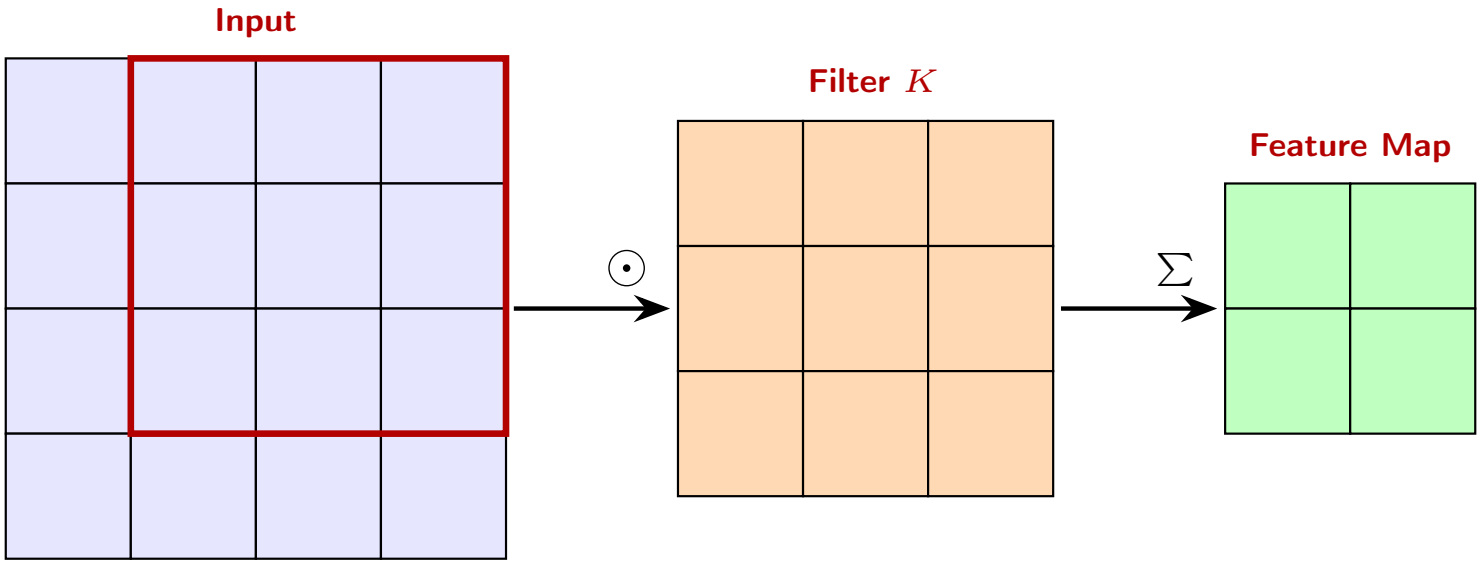
Exploit all three properties through weight sharing and local connectivity.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Convolution Operation: Visual Diagram



Sliding the filter over the input with *stride* S produces the *feature map*.

The Convolutional Layer

Convolution Equation

For input $\mathbf{I}$, kernel $\mathbf{K}$, padding P, stride S:

$$\text{out}(i,j) = \sum_{a=0}^{F-1} \sum_{b=0}^{F-1} \mathbf{K}(a,b) \cdot \mathbf{I}(i \cdot S + a, j \cdot S + b)$$

The result $\mathbf{K}(a,b)$ weights the input patch element-wise.

- Output (feature map) size: $\left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$
- W: input width, F: *filter size*, P: *padding*, S: *stride*
- Depth: one feature map per filter; C_{out} filters in total
- Parameters per filter: $F \times F \times C_{\text{in}}$ — independent of W

Key Insight: *Weight Sharing*

The same filter detects the same pattern everywhere in the image.
Parameters count drops from 150M to thousands.

The Convolutional Layer

Convolution Equation

For input $\mathbf{I}$, kernel $\mathbf{K}$, padding P, stride S:

$$\text{out}(i,j) = \sum_{a=0}^{F-1} \sum_{b=0}^{F-1} \mathbf{K}(a,b) \cdot \mathbf{I}(i \cdot S + a, j \cdot S + b)$$

The result $\mathbf{K}(a,b)$ weights the input patch element-wise.

- Output (feature map) size: $\left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$
- W: input width, F: *filter size*, P: *padding*, S: *stride*
- **Depth**: one feature map per filter; C_{out} filters in total
- **Parameters per filter**: $F \times F \times C_{\text{in}}$ — independent of W

Key Insight: *Weight Sharing*

The same filter detects the same pattern everywhere in the image.
Parameters count drops from 150M to thousands.

The Convolutional Layer

Convolution Equation

For input $\mathbf{I}$, kernel $\mathbf{K}$, padding P, stride S:

$$\text{out}(i,j) = \sum_{a=0}^{F-1} \sum_{b=0}^{F-1} \mathbf{K}(a,b) \cdot \mathbf{I}(i \cdot S + a, j \cdot S + b)$$

The result $\mathbf{K}(a,b)$ weights the input patch element-wise.

- Output (feature map) size: $\left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$
- W: input width, F: *filter size*, P: *padding*, S: *stride*
- **Depth**: one feature map per filter; C_{out} filters in total
- **Parameters per filter**: $F \times F \times C_{\text{in}}$ — independent of W

Key Insight: *Weight Sharing*

The same filter detects the same pattern everywhere in the image.
Parameters count drops from 150M to thousands.

The Convolutional Layer

Convolution Equation

For input $\mathbf{I}$, kernel $\mathbf{K}$, padding P, stride S:

$$\text{out}(i,j) = \sum_{a=0}^{F-1} \sum_{b=0}^{F-1} \mathbf{K}(a,b) \cdot \mathbf{I}(i \cdot S + a, j \cdot S + b)$$

The result $\mathbf{K}(a,b)$ weights the input patch element-wise.

- Output (feature map) size: $\left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$
- W: input width, F: *filter size*, P: *padding*, S: *stride*
- **Depth**: one feature map per filter; C_{out} filters in total
- **Parameters per filter**: $F \times F \times C_{\text{in}}$ — independent of W

Key Insight: *Weight Sharing*

The same filter detects the same pattern everywhere in the image.
Parameters count drops from 150M to thousands.

Pooling Layers

Max Pooling

- Takes maximum in 2×2 window
- Preserves strongest activation
- Provides translation invariance
- Standard in classification nets

Average Pooling

- Takes mean in window
- Smoother; used in deeper nets
- **Global Average Pooling (GAP)**: collapse entire feature map to one value per channel
- Replaces FC layers in modern nets

Pooling Operation: Example

- **Max Pooling**: $\text{pool}(a, b, c, d) = \max(a, b, c, d)$ on 2×2 window
- **Stride**: typically *same as pool size* to avoid overlap
- **Output size**: $\frac{\text{input_size} - \text{pool_size}}{\text{stride}} + 1$

Pooling Layers

Max Pooling

- Takes maximum in 2×2 window
- Preserves strongest activation
- Provides translation invariance
- Standard in classification nets

Average Pooling

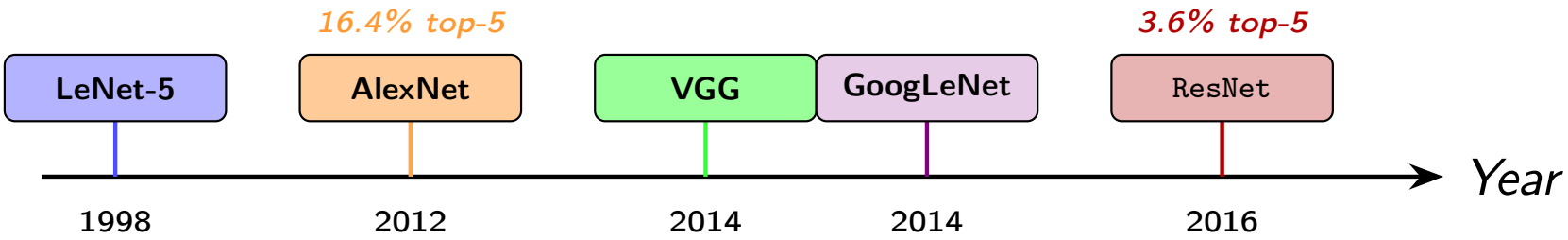
- Takes mean in window
- Smoother; used in deeper nets
- **Global Average Pooling (GAP)**: collapse entire feature map to one value per channel
- Replaces FC layers in modern nets

Pooling Operation: Example

- **Max Pooling**: $\text{pool}(a, b, c, d) = \max(a, b, c, d)$ on 2×2 window
- **Stride**: typically *same as pool size* to avoid overlap
- **Output size**: $\dfrac{\text{input_size} - \text{pool_size}}{\text{stride}} + 1$

Classic CNN Architectures

- LeNet-5 (*LeCun*, 1998): first practical CNN for digit recognition
- AlexNet (*Krizhevsky*, 2012): ImageNet breakthrough; ReLU, Dropout, *GPU training*
- VGG-16/19 (*Simonyan*, 2014): stacked 3×3 *convolutions*; very deep and clean design
- GoogLeNet/Inception (*Szegedy*, 2014): Inception modules with 1×1 convolutions
- ResNet (*He*, 2016): skip connections; 152 layers; still a strong baseline



Each milestone brought lower error and deeper networks





Data Augmentation

Why It Matters

Training deep CNNs from scratch requires large datasets. Augmentation artificially multiplies the effective training set size.





Data Augmentation

Why It Matters

Training deep CNNs from scratch requires large datasets. Augmentation artificially multiplies the effective training set size.

- **Standard**: *random crop*, *horizontal flip*, *colour jitter* (brightness/contrast/saturation)
- **Advanced**: `RandAugment` (learns augmentation policy), `AutoAugment`
- `MixUp`: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$; blends *images and labels*
- `CutMix`: paste random patch from one image into another
- **Critical rule**: augmentation applied to training only; *validation and test* use deterministic *centre crop* + *normalisation*





Data Augmentation

Why It Matters

Training deep CNNs from scratch requires large datasets. Augmentation artificially multiplies the effective training set size.

- **Standard**: *random crop*, *horizontal flip*, *colour jitter* (brightness/contrast/saturation)
- **Advanced**: `RandAugment` (learns augmentation policy), `AutoAugment`
- `MixUp`: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$; blends *images and labels*
- `CutMix`: paste random patch from one image into another
- **Critical rule**: augmentation applied to training only; *validation and test* use deterministic *centre crop* + *normalisation*





Data Augmentation

Why It Matters

Training deep CNNs from scratch requires large datasets. Augmentation artificially multiplies the effective training set size.

- **Standard**: *random crop*, *horizontal flip*, *colour jitter* (brightness/contrast/saturation)
- **Advanced**: `RandAugment` (learns augmentation policy), `AutoAugment`
- `MixUp`: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$; blends *images and labels*
- `CutMix`: paste random patch from one image into another
- **Critical rule**: augmentation applied to training only; *validation and test* use deterministic *centre crop* + *normalisation*





Data Augmentation

Why It Matters

Training deep CNNs from scratch requires large datasets. Augmentation artificially multiplies the effective training set size.

- **Standard**: *random crop*, *horizontal flip*, *colour jitter* (brightness/contrast/saturation)
- **Advanced**: `RandAugment` (learns augmentation policy), `AutoAugment`
- `MixUp`: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$; blends *images and labels*
- `CutMix`: paste random patch from one image into another
- **Critical rule**: augmentation applied to training only; *validation and test* use deterministic *centre crop* + *normalisation*





Data Augmentation

Why It Matters

Training deep CNNs from scratch requires large datasets. Augmentation artificially multiplies the effective training set size.

- **Standard**: *random crop*, *horizontal flip*, *colour jitter* (brightness/contrast/saturation)
- **Advanced**: `RandAugment` (learns augmentation policy), `AutoAugment`
- `MixUp`: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$; blends *images and labels*
- `CutMix`: paste random patch from one image into another
- **Critical rule**: augmentation applied to training only; *validation and test* use deterministic *centre crop* + *normalisation*





Receptive Field and Depth

Receptive Field

The region of the input that influences a given output neuron. **Deep networks** accumulate large receptive fields without large kernels.

- Two stacked 3×3 *conv* layers have the same receptive field as one 5×5 but with fewer parameters and two nonlinearities
- VGG insight: stack small kernels instead of using large ones
- Dilated (*atrous*) convolutions: expand receptive field without stride
- Depthwise separable convolutions: MobileNet family; reduce parameters 8×





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Receptive Field and Depth

Receptive Field

The region of the input that influences a given output neuron. **Deep networks** accumulate large receptive fields without large kernels.

- Two stacked 3×3 *conv* layers have the same receptive field as one 5×5 but with fewer parameters and two nonlinearities
- `VGG` insight: stack small kernels instead of using large ones
- Dilated (*atrous*) convolutions: expand receptive field without stride
- Depthwise separable convolutions: `MobileNet` family; reduce parameters $8\times$





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Receptive Field and Depth

Receptive Field

The region of the input that influences a given output neuron. **Deep networks** accumulate large receptive fields without large kernels.

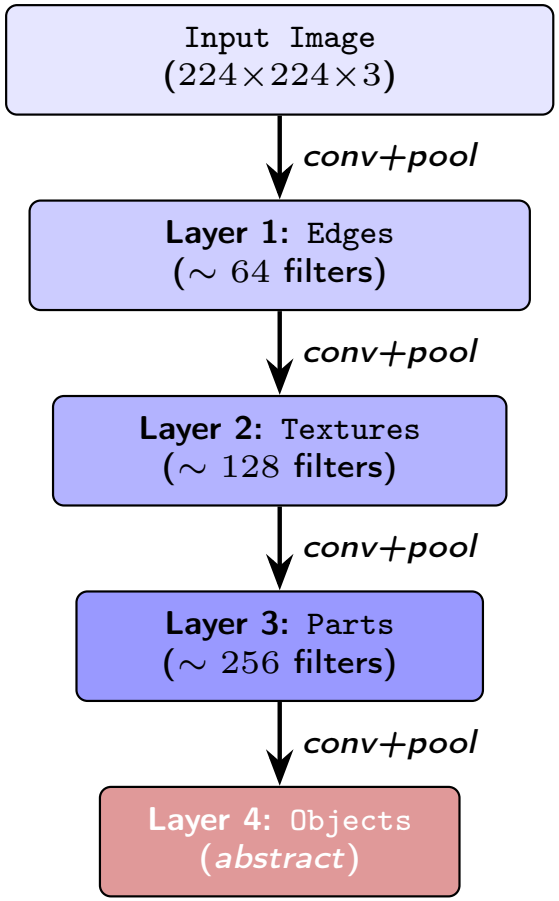
- Two stacked 3×3 *conv* layers have the same receptive field as one 5×5 but with fewer parameters and two nonlinearities
- `VGG` insight: stack small kernels instead of using large ones
- Dilated (*atrous*) convolutions: expand receptive field without stride
- Depthwise separable convolutions: `MobileNet` family; reduce parameters $8\times$





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

CNN *Feature Hierarchy*: Visual Progression



Increasing abstraction, decreasing spatial size, increasing semantic meaning

Each CNN layer learns hierarchical features building towards classification





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Example: Training Deep vs Shallow Networks (CNN Edition)

- **Dataset**: `CIFAR-10` ($60,000$ images, 32×32, 10 classes)
- **Baseline**: simple *3-layer* `CNN` (no tricks)
- **Experiment A**: add `BatchNorm` and `Dropout`
- **Experiment B**: replace plain `CNN` with `ResNet-20`
- **Experiment C**: add `RandAugment` data augmentation
- **Target**: `ResNet-20` + augmentation reaches $\approx 91\%$ *accuracy*
- **Workshop connection**: `CNN` backbones for *Challenge 7 (Transfer Learning)*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Example: Training Deep vs Shallow Networks (CNN Edition)

- **Dataset**: `CIFAR-10` ($60,000$ images, 32×32, 10 classes)
- **Baseline**: simple *3-layer* `CNN` (no tricks)
- **Experiment A**: add `BatchNorm` and `Dropout`
- **Experiment B**: replace plain `CNN` with `ResNet-20`
- **Experiment C**: add `RandAugment` data augmentation
- **Target**: `ResNet-20` + augmentation reaches $\approx 91\%$ *accuracy*
- **Workshop connection**: `CNN` backbones for *Challenge 7 (Transfer Learning)*





Practical Example: Training Deep vs Shallow Networks (CNN Edition)

- **Dataset**: `CIFAR-10` ($60,000$ images, 32×32, 10 classes)
- **Baseline**: simple *3-layer* `CNN` (no tricks)
- **Experiment A**: add `BatchNorm` and `Dropout`
- **Experiment B**: replace plain `CNN` with `ResNet-20`
- **Experiment C**: add `RandAugment` data augmentation
- **Target**: `ResNet-20` + augmentation reaches $\approx 91\%$ *accuracy*
- **Workshop connection**: `CNN` backbones for *Challenge 7 (Transfer Learning)*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1 Deep Learning Foundations

2 Convolutional Neural Networks

3 Sequence Models & Transformers

4 Unsupervised & Representation Learning





Why Sequential Models?

The Temporal Dimension

Many *real-world* data sources are ordered sequences where context matters.

- Language: "*The bank by the river*" vs. "*The bank in town*" — order disambiguates
- Time series: *stock prices, sensor readings, audio signals*
- Variable length: different sentences, different signal durations
- A fixed-size MLP cannot capture order or variable-length naturally

Solution Landscape

`RNN/LSTM/GRU` $\rightarrow$ `Attention` $\rightarrow$ `Transformer` $\rightarrow$ *Foundation Models* (`BERT, GPT`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Why Sequential Models?

The Temporal Dimension

Many *real-world* data sources are ordered sequences where context matters.

- **Language**: "*The bank by the river*" vs. "*The bank in town*" — order disambiguates
- **Time series**: *stock prices*, *sensor readings*, *audio signals*
- **Variable length**: different sentences, different signal durations
- A fixed-size `MLP` cannot capture order or variable length natively

Solution Landscape

`RNN/LSTM/GRU` → `Attention` → `Transformer` → *Foundation Models* (`BERT`, `GPT`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Why Sequential Models?

The Temporal Dimension

Many *real-world* data sources are ordered sequences where context matters.

- **Language**: "*The bank by the river*" vs. "*The bank in town*" — order disambiguates
- **Time series**: *stock prices*, *sensor readings*, *audio signals*
- **Variable length**: different sentences, different signal durations
- A fixed-size `MLP` cannot capture order or variable length natively

Solution Landscape

`RNN/LSTM/GRU` → `Attention` → `Transformer` → *Foundation Models* (`BERT, GPT`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Why Sequential Models?

The Temporal Dimension

Many *real-world* data sources are ordered sequences where context matters.

- **Language**: "*The bank by the river*" vs. "*The bank in town*" — order disambiguates
- **Time series**: *stock prices*, *sensor readings*, *audio signals*
- **Variable length**: different sentences, different signal durations
- A fixed-size `MLP` cannot capture order or variable length natively

Solution Landscape

`RNN/LSTM/GRU` → `Attention` → `Transformer` → *Foundation Models* (`BERT, GPT`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Why Sequential Models?

The Temporal Dimension

Many *real-world* data sources are ordered sequences where context matters.

- **Language**: "*The bank by the river*" vs. "*The bank in town*" — order disambiguates
- **Time series**: *stock prices*, *sensor readings*, *audio signals*
- **Variable length**: different sentences, different signal durations
- A fixed-size `MLP` cannot capture order or variable length natively

Solution Landscape

`RNN/LSTM/GRU` → `Attention` → `Transformer` → *Foundation Models* (`BERT, GPT`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Why Sequential Models?

The Temporal Dimension

Many *real-world* data sources are ordered sequences where context matters.

- **Language**: "*The bank by the river*" vs. "*The bank in town*" — order disambiguates
- **Time series**: *stock prices*, *sensor readings*, *audio signals*
- **Variable length**: different sentences, different signal durations
- A fixed-size `MLP` cannot capture order or variable length natively

Solution Landscape

`RNN/LSTM/GRU` → `Attention` → `Transformer` → *Foundation Models* (`BERT`, `GPT`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Recurrent Neural Networks (RNN)

Recurrent Update

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

- Weight sharing across time: same $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$ for *every step*
- Hidden state $\mathbf{h}_t$ is the memory of *past inputs*
- Supports variable-length sequences by design
- **Problem**: BPTT (*Backprop Through Time*) multiplies the same Jacobian L times $\rightarrow$ vanishing or exploding gradients for long sequences ($L > 20$)





Recurrent Neural Networks (RNN)

Recurrent Update

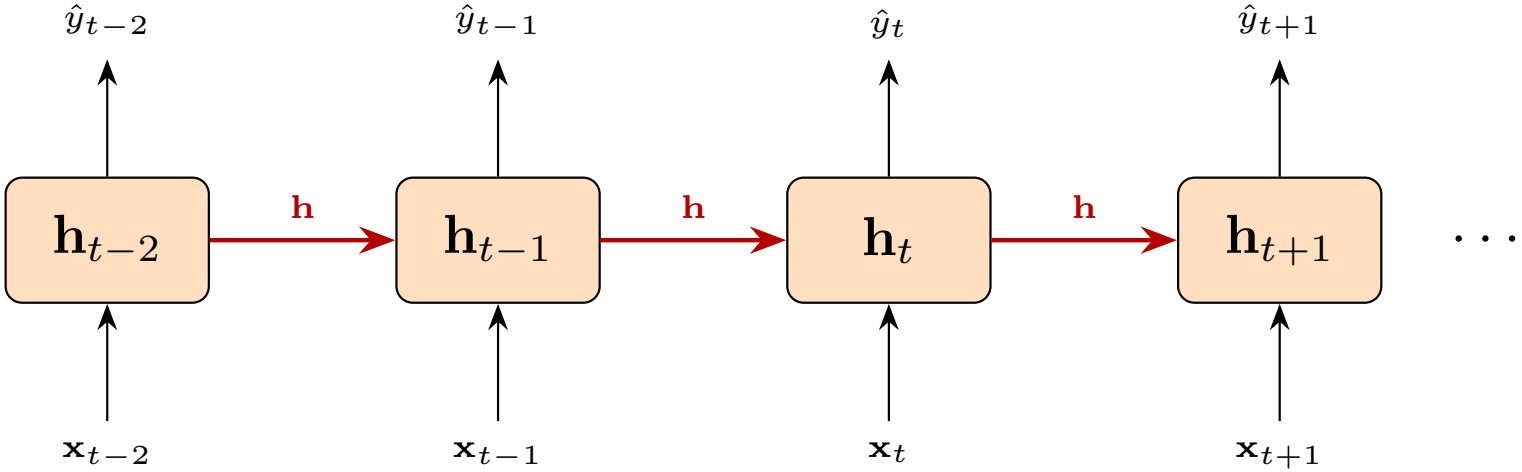
$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

- Weight sharing across time: same $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$ for *every step*
- Hidden state $\mathbf{h}_t$ is the memory of *past inputs*
- Supports variable-length sequences by design
- **Problem**: `BPTT` (*Backprop Through Time*) multiplies the same Jacobian L times $\rightarrow$ vanishing or exploding gradients for long sequences ($L > 20$)





RNN Unrolled: Information Flow



The *Backpropagation Through Time* (BPTT) Problem

As gradients flow backward through L time steps, they are multiplied by the recurrent Jacobian L times. This causes exponential scaling.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Long Short-Term Memory (*LSTM*)

Gated Cell (Hochreiter & Schmidhuber, 1997)

- **Forget gate**: $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$
- **Input gate**: $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
- **Cell state**: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$

- Cell state $\mathbf{c}_t$: *long-term memory*; gradient highway
- Hidden state $\mathbf{h}_t$: *short-term*; passed to the next step and the output
- **Forget gate**: what to erase from memory
- `GRU` (Cho *et al.*, 2014): simplified `LSTM` with *two gates* — similar performance, fewer parameters

Core idea

`LSTM` is a controlled memory cell: it learns when to *remember*, when to *forget*, and when to *expose state*.

Long Short-Term Memory (*LSTM*)

Gated Cell (Hochreiter & Schmidhuber, 1997)

- **Forget gate**: $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$
- **Input gate**: $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
- **Cell state**: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$

- Cell state $\mathbf{c}_t$: *long-term memory*; gradient highway
- Hidden state $\mathbf{h}_t$: *short-term*; passed to the next step and the output
- **Forget gate**: what to erase from memory
- `GRU` (Cho *et al.*, 2014): simplified `LSTM` with *two gates* — similar performance, fewer parameters

Core idea

`LSTM` is a controlled memory cell: it learns when to *remember*, when to *forget*, and when to *expose state*.

Long Short-Term Memory (*LSTM*)

Gated Cell (Hochreiter & Schmidhuber, 1997)

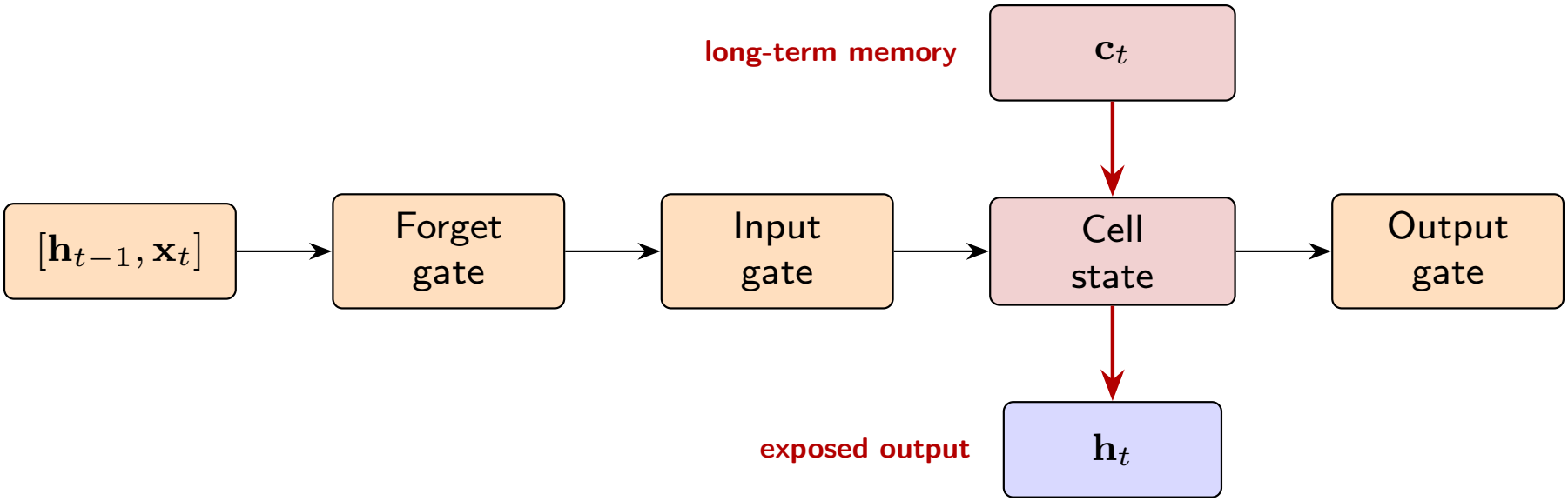
- **Forget gate**: $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$
- **Input gate**: $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
- **Cell state**: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$

- Cell state $\mathbf{c}_t$: *long-term memory*; gradient highway
- Hidden state $\mathbf{h}_t$: *short-term*; passed to the next step and the output
- **Forget gate**: what to erase from memory
- `GRU` (Cho *et al.*, 2014): simplified `LSTM` with *two gates* — similar performance, fewer parameters

Core idea

`LSTM` is a controlled memory cell: it learns when to *remember*, when to *forget*, and when to *expose state*.

LSTM Cell: Gate Diagram



Intuition

The *forget gate* erases outdated information, the *input gate* writes new information, the *cell state* acts as a stable memory highway, and the *output gate* exposes selected information to the next step.





LSTM vs GRU: Trade-offs

LSTM (3 gates)

- More expressive: separate *cell state* and *hidden state*
- Slower and *more parameters* ($\sim 4\times$ RNN)
- **Better** for very long dependencies ($100+$ steps)
- Standard choice for *sequence-to-sequence* and *language modelling*

GRU (2 gates)

- Simpler: combines cell and hidden states into single state
- Faster and *fewer parameters* ($\sim 3\times$ RNN)
- Similar *performance* in practice; often preferred for *efficiency*
- Easier to understand and debug than LSTM





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Choice: *When to Use What*

Decision Framework

- **Short sequences** (< 20 steps), *efficiency matters*: `GRU` or even simple `RNN`
- **Long sequences** (> 50 steps), *capture complex patterns*: `LSTM`
- **Language tasks** (translation, QA, generation): `LSTM` or `Transformer`
- **Real-time inference** (mobile, edge): `GRU` strongly preferred
- **No clear winner**: try both `LSTM` and `GRU`, compare on validation set

Modern Trend

`Transformers` have largely replaced `LSTM`/`GRU` for NLP tasks due to parallelism, but `LSTMs` remain popular for *time series* and resource-constrained settings.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Choice: *When to Use What*

Decision Framework

- **Short sequences** (< 20 steps), *efficiency matters*: `GRU` or even simple `RNN`
- **Long sequences** (> 50 steps), *capture complex patterns*: `LSTM`
- **Language tasks** (translation, QA, generation): `LSTM` or `Transformer`
- **Real-time inference** (mobile, edge): `GRU` strongly preferred
- **No clear winner**: try both `LSTM` and `GRU`, compare on validation set

Modern Trend

`Transformers` have largely replaced `LSTM`/`GRU` for NLP tasks due to parallelism, but `LSTM`s remain popular for *time series* and resource-constrained settings.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Attention Mechanism

Attention (Bahdanau *et al.*, 2015)

Let the **decoder** query all *encoder hidden states* $\mathbf{h}_j$ simultaneously:

$$e_{ij} = \text{score}(\mathbf{s}_i, \mathbf{h}_j), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}, \quad \mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j$$

- α_{ij}: soft alignment weight — how much position i *attends to* j
- Abolishes the information bottleneck of the fixed-size context vector
- Made *neural machine translation* practical ($\gg 10$ words)
- **Key insight**: attention is differentiable and trainable *end-to-end*





Attention Mechanism

Attention (Bahdanau *et al.*, 2015)

Let the **decoder** query all *encoder hidden states* $\mathbf{h}_j$ simultaneously:

$$e_{ij} = \text{score}(\mathbf{s}_i, \mathbf{h}_j), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}, \quad \mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j$$

- α_{ij}: soft alignment weight — how much position i *attends to* j
- Abolishes the information bottleneck of the fixed-size context vector
- Made *neural machine translation* practical ($\gg 10$ words)
- Key insight: attention is differentiable and trainable *end-to-end*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Attention Mechanism

Attention (Bahdanau *et al.*, 2015)

Let the **decoder** query all *encoder hidden states* $\mathbf{h}_j$ simultaneously:

$$e_{ij} = \text{score}(\mathbf{s}_i, \mathbf{h}_j), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}, \quad \mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j$$

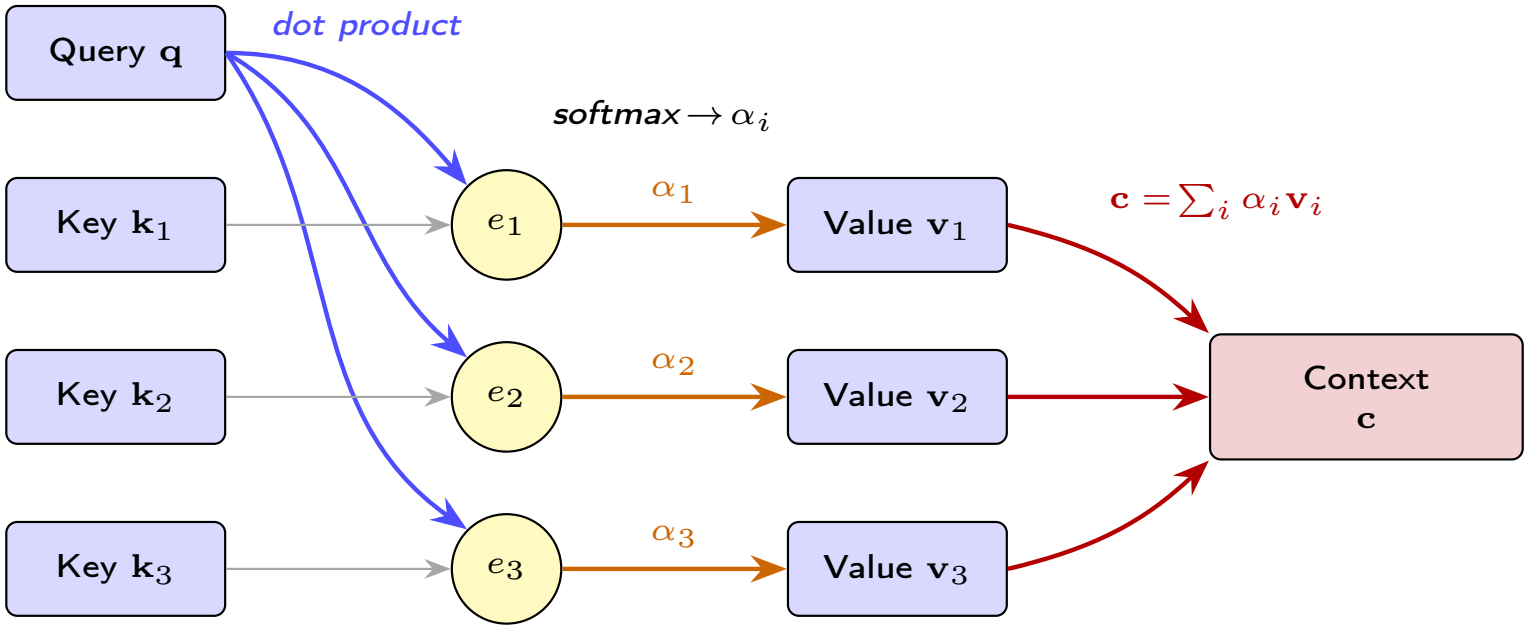
- α_{ij}: soft alignment weight — how much position i *attends to* j
- Abolishes the information bottleneck of the fixed-size context vector
- Made *neural machine translation* practical ($\gg 10$ words)
- **Key insight**: attention is differentiable and trainable *end-to-end*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Attention Mechanism: Vector View



The query determines which values contribute most to the context vector.

The Transformer

`Transformer` (Vaswani *et al.*, 2017) — "Attention Is All You Need"

No recurrence; entirely based on *self-attention* and *feed-forward* layers.

Scaled Dot-Product Self-Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$
- Multi-Head Attention: run h parallel attention heads; *concatenate* outputs
- Captures long-range dependencies in $O(1)$ layers vs. $O(n)$ for RNN
- Cost: $O(n^2)$ memory in sequence length — can be a bottleneck





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

The Transformer

`Transformer` (Vaswani *et al.*, 2017) — "Attention Is All You Need"

No recurrence; entirely based on *self-attention* and *feed-forward* layers.

Scaled Dot-Product Self-Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$
- Multi-Head Attention: run h parallel attention heads; *concatenate* outputs
- Captures long-range dependencies in $O(1)$ layers vs. $O(n)$ for RNN
- Cost: $O(n^2)$ memory in sequence length — quadratic attention





The Transformer

`Transformer` (Vaswani *et al.*, 2017) — "Attention Is All You Need"

No recurrence; entirely based on *self-attention* and *feed-forward* layers.

Scaled Dot-Product Self-Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$
- **Multi-Head Attention**: run h parallel attention heads; *concatenate* outputs
- Captures long-range dependencies in $O(1)$ layers vs. $O(n)$ for `RNN`
- Cost: $O(n^2)$ memory in sequence length — quadratic attention





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

The Transformer

`Transformer` (Vaswani *et al.*, 2017) — "Attention Is All You Need"

No recurrence; entirely based on *self-attention* and *feed-forward* layers.

Scaled Dot-Product Self-Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

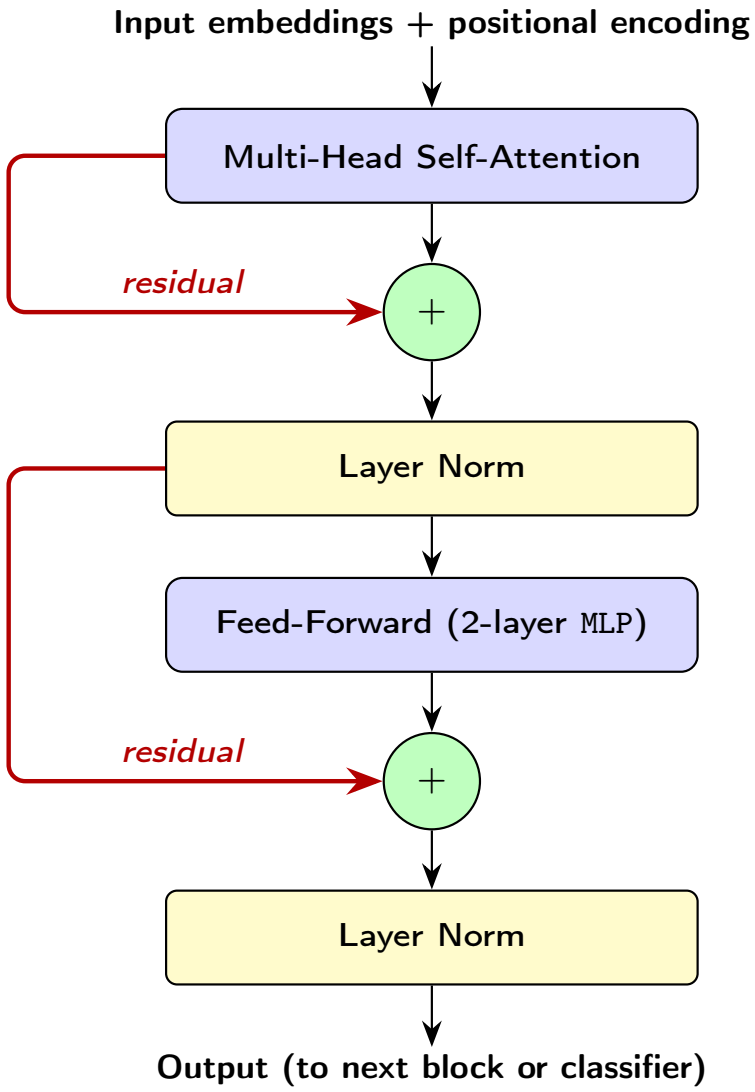
- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$
- **Multi-Head Attention**: run h parallel attention heads; *concatenate* outputs
- Captures long-range dependencies in $O(1)$ layers vs. $O(n)$ for `RNN`
- Cost: $O(n^2)$ memory in sequence length — quadratic attention





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Transformer Block: *Architecture* Diagram



Positional Encoding and Transformer Architecture

Positional Encoding

Self-attention is permutation-invariant; we must inject order information:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

`Transformer` Block (stacked N times)

1. *Multi-Head Self-Attention* (with residual + `LayerNorm`)
2. *Position-wise Feed-Forward* (with residual + `LayerNorm`)

- Encoder-Decoder: original `Transformer` (*translation*)
- Encoder-only: `BERT` (*classification*); Decoder-only: `GPT` (*generation*)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Positional Encoding and Transformer Architecture

Positional Encoding

Self-attention is permutation-invariant; we must inject order information:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

`Transformer` Block (stacked N times)

1. *Multi-Head Self-Attention* (with residual + `LayerNorm`)
2. *Position-wise Feed-Forward* (with residual + `LayerNorm`)

- Encoder-Decoder: original `Transformer` (*translation*)
- Encoder-only: `BERT` (*classification*); Decoder-only: `GPT` (*generation*)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Positional Encoding and Transformer Architecture

Positional Encoding

Self-attention is permutation-invariant; we must inject order information:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

`Transformer` Block (stacked N times)

1. *Multi-Head Self-Attention* (with residual + `LayerNorm`)
2. *Position-wise Feed-Forward* (with residual + `LayerNorm`)

- **Encoder-Decoder**: original `Transformer` (*translation*)
- **Encoder-only**: `BERT` (*classification*); **Decoder-only**: `GPT` (*generation*)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Transformer Block: Summary of Components

Why Transformers Win

1. No recurrence: compute all time steps in parallel (vs. sequential RNN)
2. Long-range interactions: direct connections between any two positions
3. Interpretable: attention weights show which tokens are *important*
4. Scalable: foundation for 100B+ parameter models (GPT-3, LLaMA)





Foundation Models: BERT and GPT

BERT (Devlin, 2019)

- Bidirectional *encoder*
- *Pretraining*: Masked LM + *Next Sentence Prediction*
- *Fine-tune* for: *classification*, *QA*, *NER*

GPT-2/3/4 (OpenAI)

- Autoregressive *decoder*
- *Pretraining*: *causal language modelling*
- *Few-shot / zero-shot* generation

Key Pattern

Pretrain on massive unlabelled data, then fine-tune on *small labelled sets* — the same paradigm as *Transfer Learning*.





Foundation Models: BERT and GPT

BERT (Devlin, 2019)

- Bidirectional *encoder*
- *Pretraining*: Masked LM + *Next Sentence Prediction*
- *Fine-tune* for: *classification*, *QA*, *NER*

GPT-2/3/4 (OpenAI)

- Autoregressive *decoder*
- *Pretraining*: *causal language modelling*
- *Few-shot* / *zero-shot* generation

Key Pattern

Pretrain on massive unlabelled data, then fine-tune on *small labelled sets* — the same paradigm as *Transfer Learning*.





Practical Example: Sentiment Classification with Transformer

- **Dataset**: IMDB reviews ($50k$, *binary sentiment*)
- Model A: *2-layer* LSTM (baseline)
- Model B: small Transformer encoder (4 heads, 2 layers)
- Model C: pre-trained bert-base-uncased fine-tuned
- Metrics: *accuracy*, *precision*, *recall* on test set
- **Expected result**: BERT $\gg$ Transformer $\geq$ LSTM with the same compute budget





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Example: Sentiment Classification with Transformer

- **Dataset**: `IMDB` reviews ($50k$, *binary sentiment*)
- **Model A**: *2-layer* `LSTM` (baseline)
- **Model B**: small `Transformer` encoder (4 heads, 2 layers)
- **Model C**: pre-trained `bert-base-uncased` fine-tuned
- **Metrics**: *accuracy*, *precision*, *recall* on test set
- **Expected result**: `BERT` $\gg$ `Transformer` $\geq$ `LSTM` with the same compute budget





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Example: Sentiment Classification with Transformer

- **Dataset**: `IMDB` reviews ($50k$, *binary sentiment*)
- **Model A**: *2-layer* `LSTM` (baseline)
- **Model B**: small `Transformer` encoder (4 heads, 2 layers)
- **Model C**: pre-trained `bert-base-uncased` fine-tuned
- **Metrics**: *accuracy*, *precision*, *recall* on test set
- **Expected result**: `BERT` $\gg$ `Transformer` $\geq$ `LSTM` with the same compute budget





Outline

1 Deep Learning Foundations

2 Convolutional Neural Networks

3 Sequence Models & Transformers

4 Unsupervised & Representation Learning





Unsupervised Learning Landscape

Unsupervised Learning Goal

Learn the underlying structure of **data** $\mathbf{x}$ without **labels** $\mathbf{y}$. Applications: *compression*, *feature discovery*, *anomaly detection*, *generation*.

- Density estimation: model the *data distribution* explicitly
- Dimensionality reduction: find compact representations (AE, PCA)
- Clustering: discover group structure (K-Means, DBSCAN — Ch. 5)
- Anomaly detection: identify outliers (OC-SVM, Isolation Forest)
- Generative modelling: synthesise new samples (GANs)





Unsupervised Learning Landscape

Unsupervised Learning Goal

Learn the underlying structure of **data** $\mathbf{x}$ without **labels** $\mathbf{y}$. Applications: *compression*, *feature discovery*, *anomaly detection*, *generation*.

- **Density estimation**: model the *data distribution* explicitly
- **Dimensionality reduction**: find compact representations (`AE`, `PCA`)
- **Clustering**: discover group structure (`K-Means`, `DBSCAN` — *Challenge 5*)
- **Anomaly detection**: identify rare, unusual samples (*Challenge 6*)
- **Generative modelling**: synthesise new samples (`GANs`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Unsupervised Learning Landscape

Unsupervised Learning Goal

Learn the underlying structure of **data** $\mathbf{x}$ without **labels** $\mathbf{y}$. Applications: *compression*, *feature discovery*, *anomaly detection*, *generation*.

- **Density estimation**: model the *data distribution* explicitly
- **Dimensionality reduction**: find compact representations (`AE`, `PCA`)
- **Clustering**: discover group structure (`K-Means`, `DBSCAN` — *Challenge 5*)
- **Anomaly detection**: identify rare, unusual samples (*Challenge 6*)
- **Generative modelling**: synthesise new samples (`GANs`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Unsupervised Learning Landscape

Unsupervised Learning Goal

Learn the underlying structure of **data** $\mathbf{x}$ without **labels** $\mathbf{y}$. Applications: *compression*, *feature discovery*, *anomaly detection*, *generation*.

- **Density estimation**: model the *data distribution* explicitly
- **Dimensionality reduction**: find compact representations (`AE`, `PCA`)
- **Clustering**: discover group structure (`K-Means`, `DBSCAN` — *Challenge 5*)
- **Anomaly detection**: identify rare, unusual samples (*Challenge 6*)
- **Generative modelling**: synthesise new samples (`GANs`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Unsupervised Learning Landscape

Unsupervised Learning Goal

Learn the underlying structure of **data** $\mathbf{x}$ without **labels** $\mathbf{y}$. Applications: *compression*, *feature discovery*, *anomaly detection*, *generation*.

- **Density estimation**: model the *data distribution* explicitly
- **Dimensionality reduction**: find compact representations (`AE`, `PCA`)
- **Clustering**: discover group structure (`K-Means`, `DBSCAN` — *Challenge 5*)
- **Anomaly detection**: identify rare, unusual samples (*Challenge 6*)
- **Generative modelling**: synthesise new samples (`GANs`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Unsupervised Learning Landscape

Unsupervised Learning Goal

Learn the underlying structure of **data** $\mathbf{x}$ without **labels** $\mathbf{y}$. Applications: *compression*, *feature discovery*, *anomaly detection*, *generation*.

- **Density estimation**: model the *data distribution* explicitly
- **Dimensionality reduction**: find compact representations (`AE`, `PCA`)
- **Clustering**: discover group structure (`K-Means`, `DBSCAN` — *Challenge 5*)
- **Anomaly detection**: identify rare, unusual samples (*Challenge 6*)
- **Generative modelling**: synthesise new samples (`GANs`)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

AutoEncoder Architecture

Definition

A *neural network* trained to reconstruct its input through a compressed bottleneck (*latent code* $\mathbf{z}$):

$$\hat{\mathbf{x}} = \text{Dec}(\text{Enc}(\mathbf{x})), \qquad \mathcal{L}_{\text{AE}} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

- **Encoder**: $\mathbf{x} \to \mathbf{z}$; dimensionality reduction
- **Decoder**: $\mathbf{z} \to \hat{\mathbf{x}}$; reconstruction
- **Bottleneck**: forces the encoder to learn a compressed representation
- `Denoising AE`: corrupt input with *noise*; train to reconstruct clean — more robust features





AutoEncoder Architecture

Definition

A *neural network* trained to reconstruct its input through a compressed bottleneck (*latent code* $\mathbf{z}$):

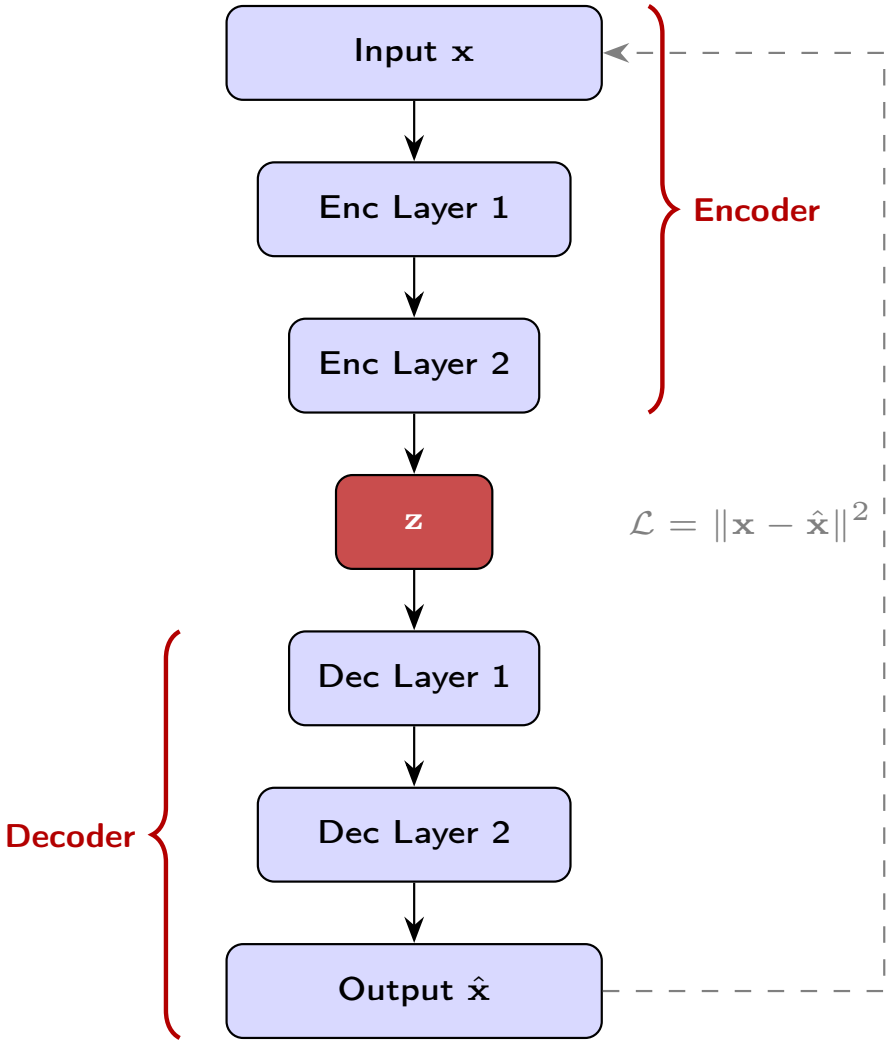
$$\hat{\mathbf{x}} = \text{Dec}(\text{Enc}(\mathbf{x})), \qquad \mathcal{L}_{\text{AE}} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

- **Encoder**: $\mathbf{x} \to \mathbf{z}$; dimensionality reduction
- **Decoder**: $\mathbf{z} \to \hat{\mathbf{x}}$; reconstruction
- **Bottleneck**: forces the encoder to learn a compressed representation
- `Denoising AE`: corrupt input with *noise*; train to reconstruct clean — more robust features





AutoEncoder: Architecture Diagram



Latent Space and Anomaly Detection

Normal training

Train `AE` only on normal samples. The model learns to compress and reconstruct *typical patterns*.

`Anomaly` Scoring Algorithm

At *inference*, anomalous samples have high reconstruction error because the `encoder` was never trained to compress them:

$$\text{score}(\mathbf{x}) = \|\mathbf{x} - \text{Dec}(\text{Enc}(\mathbf{x}))\|^2$$

Threshold at the 95th *percentile* of training set errors.

- This is the core idea of *Challenge 6*
- Compare with `Isolation Forest` baseline





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Latent Space and Anomaly Detection

Normal training

Train `AE` only on normal samples. The model learns to compress and reconstruct *typical patterns*.

`Anomaly` Scoring Algorithm

At *inference*, anomalous samples have high reconstruction error because the `encoder` was never trained to compress them:

$$\text{score}(\mathbf{x}) = \|\mathbf{x} - \text{Dec}(\text{Enc}(\mathbf{x}))\|^2$$

Threshold at the 95th *percentile* of training set errors.

- This is the core idea of *Challenge 6*
- Compare with `Isolation Forest` baseline





Latent Space and Anomaly Detection

Normal training

Train `AE` only on normal samples. The model learns to compress and reconstruct *typical patterns*.

`Anomaly` Scoring Algorithm

At *inference*, anomalous samples have high reconstruction error because the `encoder` was never trained to compress them:

$$\text{score}(\mathbf{x}) = \|\mathbf{x} - \text{Dec}(\text{Enc}(\mathbf{x}))\|^2$$

Threshold at the 95th *percentile* of training set errors.

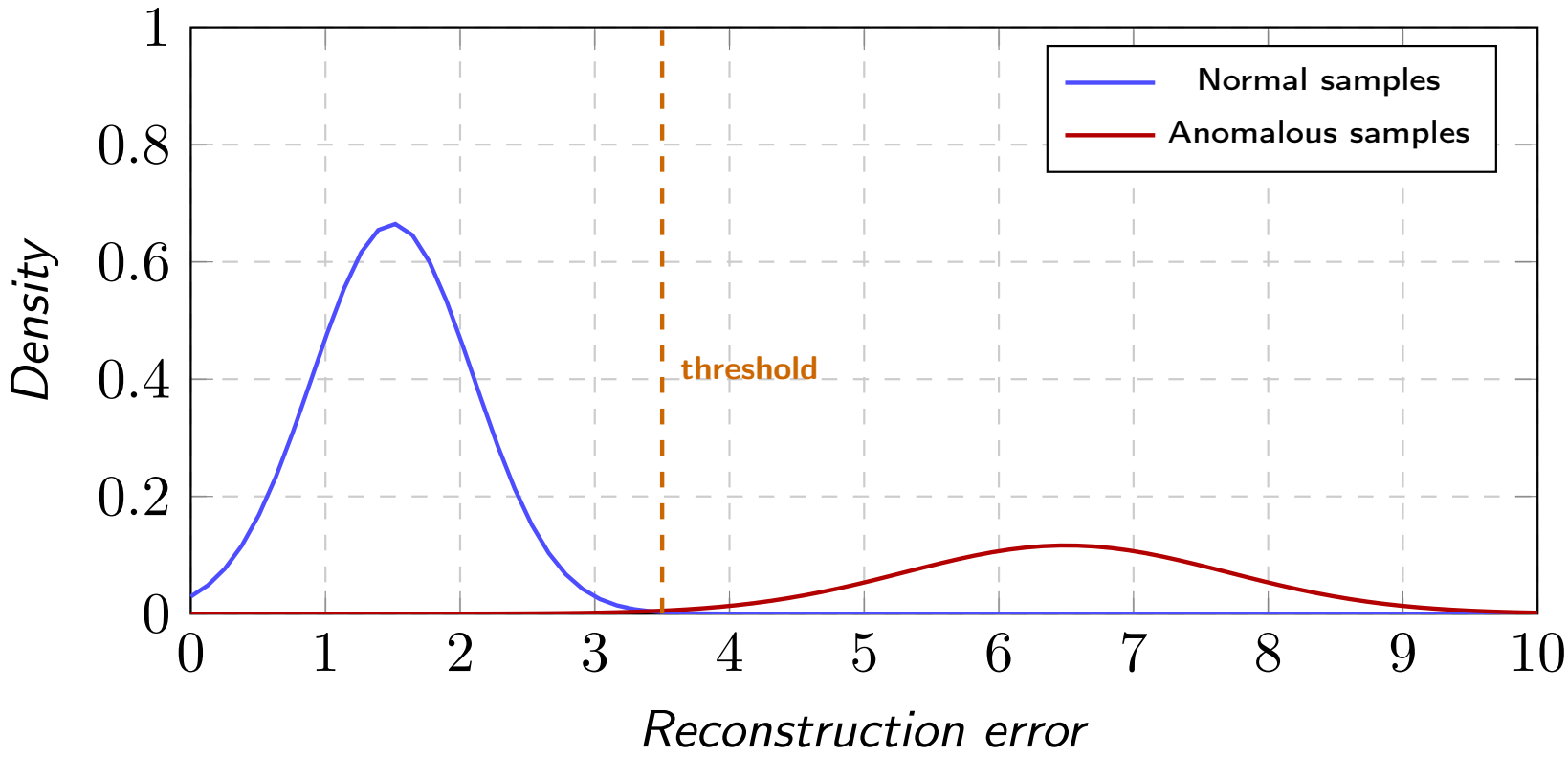
- This is the core idea of *Challenge 6*
- Compare with `Isolation Forest` baseline





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Anomaly Detection: Reconstruction Error Distribution



Samples with error above the threshold are flagged as anomalies





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Variational AutoEncoder (VAE)

Key Difference from AE (Kingma & Welling, 2014)

The encoder outputs a distribution over $\mathbf{z}$: $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ rather than a single point $\mathbf{z}$.

ELBO Loss

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\textit{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\textit{regularisation}}$$

- **Reparameterisation trick:** $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ — makes backpropagation through *sampling* possible
- KL term: pushes $q_\phi(\mathbf{z}|\mathbf{x})$ towards $\mathcal{N}(\mathbf{0}, \mathbf{I})$, creates a smooth, continuous latent space





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Variational AutoEncoder (VAE)

Key Difference from AE (Kingma & Welling, 2014)

The encoder outputs a distribution over $\mathbf{z}$: $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ rather than a single point $\mathbf{z}$.

ELBO Loss

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\textit{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\textit{regularisation}}$$

- Reparameterisation trick: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ $\longrightarrow$ makes backpropagation through *sampling* possible
- KL term: pushes $q_\phi(\mathbf{z}|\mathbf{x})$ towards $\mathcal{N}(\mathbf{0}, \mathbf{I})$; creates a smooth, continuous latent space





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Variational AutoEncoder (VAE)

Key Difference from AE (Kingma & Welling, 2014)

The encoder outputs a distribution over $\mathbf{z}$: $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ rather than a single point $\mathbf{z}$.

ELBO Loss

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\textit{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\textit{regularisation}}$$

- **Reparameterisation trick**: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ — makes backpropagation through *sampling* possible
- KL term: pushes $q_\phi(\mathbf{z}|\mathbf{x})$ towards $\mathcal{N}(\mathbf{0}, \mathbf{I})$; creates a smooth, continuous latent space





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Variational AutoEncoder (VAE)

Key Difference from AE (Kingma & Welling, 2014)

The encoder outputs a distribution over $\mathbf{z}$: $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ rather than a single point $\mathbf{z}$.

ELBO Loss

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\textit{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\textit{regularisation}}$$

- **Reparameterisation trick**: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ — makes backpropagation through *sampling* possible
- KL term: pushes $q_\phi(\mathbf{z}|\mathbf{x})$ towards $\mathcal{N}(\mathbf{0}, \mathbf{I})$; creates a smooth, continuous latent space





t-SNE and UMAP for Visualisation

`t-SNE` (van der Maaten, 2008)

- Converts *distances* to *conditional probabilities*
- Minimises `KL divergence` between high-D and 2-D distributions
- Preserves local structure
- Perplexity: neighbourhood size (5–50)
- Slow on large datasets

`UMAP` (McInnes, 2018)

- *Topological data analysis* foundations
- Preserves both local and global structure
- Much faster; scales to millions
- `n_neighbors`, `min_dist` control trade-off
- Preferred in practice

Visualisation Landscape

`PCA`: *fast, linear, preserves global structure*; `t-SNE`: *slower, nonlinear, preserves local structure*; `UMAP`: *balanced, preserves both local and global.*





t-SNE and UMAP for Visualisation

`t-SNE` (van der Maaten, 2008)

- Converts *distances* to *conditional probabilities*
- Minimises `KL divergence` between high-D and 2-D distributions
- Preserves local structure
- Perplexity: neighbourhood size (5–50)
- Slow on large datasets

`UMAP` (McInnes, 2018)

- *Topological data analysis* foundations
- Preserves both local and global structure
- Much faster; scales to millions
- `n_neighbors`, `min_dist` control trade-off
- Preferred in practice

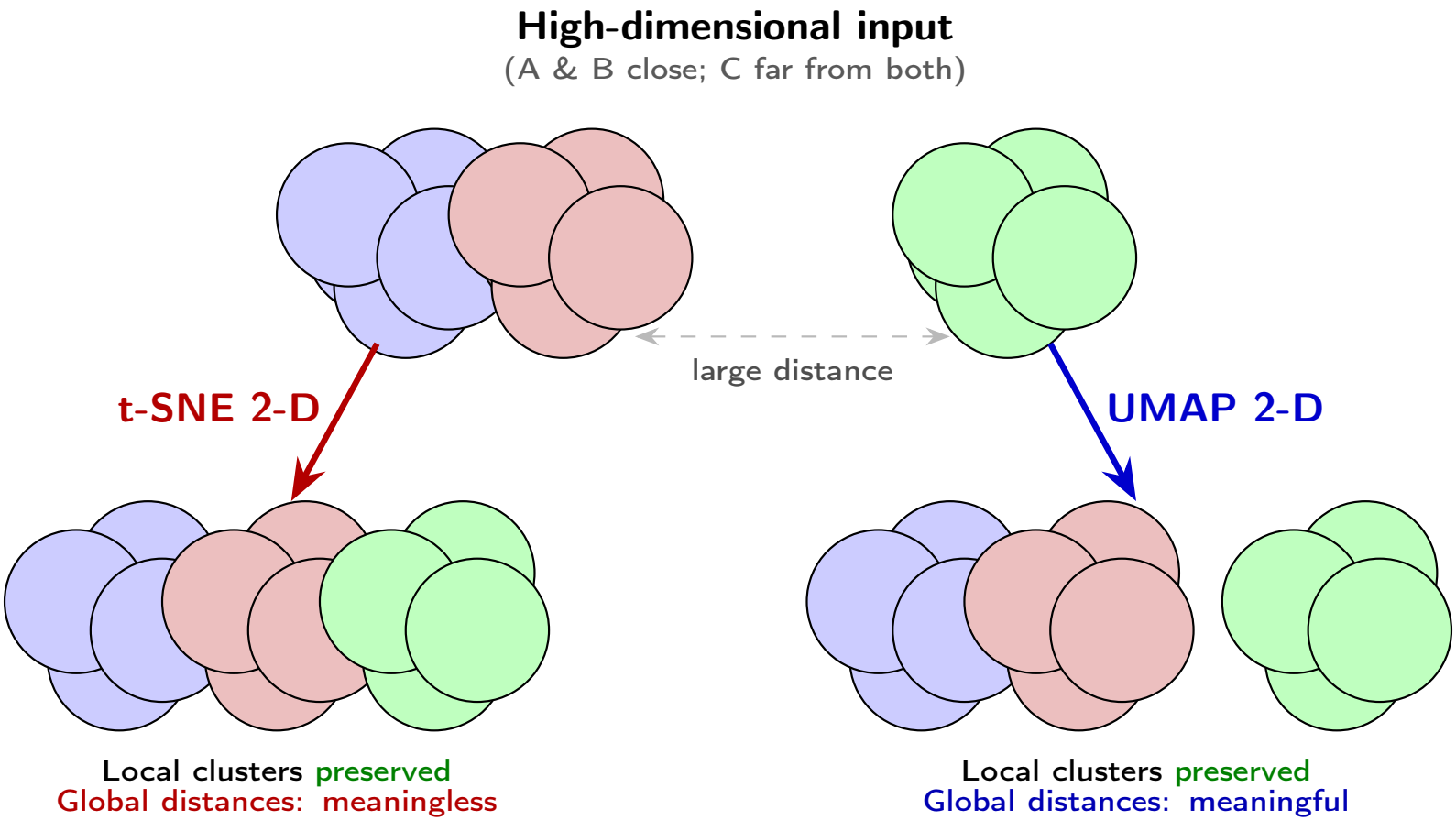
Visualisation Landscape

`PCA`: *fast*, *linear*, *preserves global structure*; `t-SNE`: *slower*, *nonlinear*, *preserves local structure*; `UMAP`: *balanced*, *preserves both local and global*.





t-SNE vs UMAP: Concept Diagram



Challenge 5 & 6 Connection

Challenge 5 — Clustering

Apply `K-Means`, `DBSCAN`, and `Hierarchical Clustering` to an assigned U.S. open government *dataset*. Visualise with `PCA` and `t-SNE/UMAP`.

Challenge 6 — `AutoEncoders` & Anomaly Detection

- Re-use the same preprocessed feature matrix from *Challenge 5*
- Train a standard `AE` and a `VAE`; compare *reconstruction error histograms*
- Visualise `VAE` *latent space* with `t-SNE` and `UMAP`
- Colour by *Challenge 5* cluster labels to compare paradigms
- Baseline: `Isolation Forest`; compare *Spearman rank correlation* with `AE` errors





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Challenge 5 & 6 Connection

Challenge 5 — Clustering

Apply `K-Means`, `DBSCAN`, and `Hierarchical Clustering` to an assigned U.S. open government *dataset*. Visualise with `PCA` and `t-SNE/UMAP`.

Challenge 6 — `AutoEncoders` & Anomaly Detection

- Re-use the same preprocessed feature matrix from *Challenge 5*
- Train a standard `AE` and a `VAE`; compare *reconstruction error histograms*
- Visualise `VAE` *latent space* with `t-SNE` and `UMAP`
- Colour by *Challenge 5* cluster labels to compare paradigms
- Baseline: `Isolation Forest`; compare *Spearman rank correlation* with `AE` errors





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Example: AE Anomaly Detection on Tabular Data

- **Dataset**: *Credit card fraud* (Kaggle), $284k$ transactions, 0.17% fraud
- **Protocol**: train AE on normal transactions only
- **Step 1**: *scatter plot* of *reconstruction errors* (*normal vs. fraud*)
- **Step 2**: threshold at 95th *percentile*; compute *precision*/*recall*
- **Step 3**: project *latent space* with UMAP; colour by *fraud label*
- **Expected**: fraud points cluster in high-error regions and appear as *outliers* in the 2-D projection





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Practical Example: AE Anomaly Detection on Tabular Data

- **Dataset**: *Credit card fraud* (Kaggle), $284k$ transactions, 0.17% fraud
- **Protocol**: train AE on normal transactions only
- **Step 1**: *scatter plot* of *reconstruction errors* (*normal vs. fraud*)
- **Step 2**: threshold at 95th *percentile*; compute *precision*/*recall*
- **Step 3**: project *latent space* with UMAP; colour by *fraud label*
- **Expected**: fraud points cluster in high-error regions and appear as *outliers* in the 2-D projection





Practical Example: AE Anomaly Detection on Tabular Data

- **Dataset**: *Credit card fraud* (Kaggle), $284k$ transactions, 0.17% fraud
- **Protocol**: train AE on normal transactions only
- **Step 1**: *scatter plot* of *reconstruction errors* (*normal vs. fraud*)
- **Step 2**: threshold at 95th *percentile*; compute *precision/recall*
- **Step 3**: project *latent space* with UMAP; colour by *fraud label*
- **Expected**: fraud points cluster in high-error regions and appear as *outliers* in the 2-D projection





Outline

1 Deep Learning Foundations

2 Convolutional Neural Networks

3 Sequence Models & Transformers

4 Unsupervised & Representation Learning





Thanks!

Questions?



Repo: *https://github.com/EngAndres/ud-public/tree/main/courses/machine-learning*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS